

UAI301 Object-Oriented Programming

Lab Assignment 5.2

Topic: Encapsulation (Access Specifiers - Public, Private, Protected, Creating Child Class, Getters and Setters, Data Hiding and Abstraction)

Case Study 1: FinTech & Banking (Account Security)

A software engineering team at a bank is building an Account system. Financial balances must never be modified directly from outside the class to prevent unauthorized tampering or invalid inputs (e.g., negative balances).

Write a Python program for a BankAccount class:

- Declare `__balance` as a private attribute and initialize it via `__init__(self, owner, initial_balance)`.
- Implement a public method `deposit(amount)` that only adds money if the amount is greater than 0.
- Implement a public method `withdraw(amount)` that subtracts money only if the balance is sufficient.
- Implement a getter method `get_balance()` to view the current balance.

Test Case: Attempt to directly overwrite `account.__balance = 1000000` from the main program, and verify using `get_balance()` that the private attribute remained untampered.

Case Study 2: E-Commerce Order Processing (Protected Logistics)

An e-commerce platform manages shipments using a base class `Order` and a specialized subclass `ExpressOrder`. Shipping routing information needs to be shared within the shipping pipeline (child classes) but hidden from public API consumers.

Write a Python program demonstrating protected members in inheritance:

- Create a parent class `Order` with a protected attribute `_shipping_status` initialized to "Processing".
- Create a subclass `ExpressOrder` that inherits from `Order`.
- In `ExpressOrder`, write a method `expedite_shipping()` that modifies `_shipping_status` to "In Transit - Priority".
- Implement a public `track_order()` method in `ExpressOrder` that returns the status.
- Demonstrate instantiating an `ExpressOrder`, calling `expedite_shipping()`, and tracking the result.

Case Study 3: Healthcare Records & Patient Data (Flexible Getters/Setters)

A medical records application manages patient vitals. The team wants to enforce validation on patient age updates without being forced to use rigid `get_` and `set_` naming conventions.

Write a Python program:

- Create a `Patient` class with private attributes `__name` and `__age`.

- Write custom access methods that **do not** use standard `get_/set_` prefixes (e.g., use `fetch_age()` and `update_age(new_age)`).
- Inside `update_age()`, ensure the age is between 0 and 120. If an invalid age is provided, display an error message and cancel the update.
- Test updating the patient's age with both a valid value (35) and an invalid value (150).

Case Study 4: Enterprise Payroll System (Preventing Subclass Overwrites)

A parent company designs a core `BasePayroll` engine where salary calculations use a internal formula variable `__tax_rate`. A third-party developer extending this class in a subclass `RegionalPayroll` accidentally names a variable `__tax_rate` as well.

Write a Python program demonstrating how double underscores prevent variable overwriting in inheritance:

- Create a base class `BasePayroll` with a private attribute `__tax_rate = 0.20` and a method `calculate_base_tax(salary)` that uses `self.__tax_rate`.
- Create a child class `RegionalPayroll(BasePayroll)` that sets `__tax_rate = 0.05` in its `__init__`.
- Demonstrate that calling `calculate_base_tax()` on an instance of `RegionalPayroll` still uses the parent's tax rate (0.20), proving that Python's name mangling kept both variables completely separate.

Case Study 5: Smart Home IoT Device (Controlled State Changes)

A smart home thermostat manages ambient room temperature. The target temperature can only be adjusted within safe hardware limits (**16°C** to **30°C**) to avoid damaging the HVAC compressor.

Write a Python program for a `Thermostat` class:

- Initialize `__target_temp` as a private variable default to 22.
- Implement a method `set_temperature(temp)` that validates whether **16 ≤ temp ≤ 30**. If valid, update the private attribute; otherwise, raise a user warning.
- Implement a method `get_temperature()` to return the current setting.
- Demonstrate attempting to set the temperature to **10°C** and **24°C**.

Case Study 6: Corporate HR Portal (Employee Directory Access Levels)

An internal HR dashboard exposes public employee names, protected team project assignments, and private salary details across various company modules.

Write a Python program:

- Create an `Employee` class containing:
 - Public: `name`
 - Protected: `_project`

- Private: `__salary`
- Create a child class `Developer` that inherits from `Employee` and includes a method `get_dev_summary()` accessing both `name` and `_project`.
- In the main program execution:
 - i. Access `name` and `_project` directly from an instance.
 - ii. Attempt to access `__salary` directly and handle the resulting `AttributeError` with a `try...except` block.
 - iii. Extract `__salary` using its mangled attribute name `_Employee__salary`.

Case Study 7: Digital Wallet Service (Pin-Protected Transactions)

A payment application (`DigitalWallet`) allows balance transfers only if the correct 4-digit PIN is provided.

Write a Python program:

- Private attributes: `__balance` and `__pin`.
- Implement a method `verify_pin(entered_pin)` that returns `True` if the entered PIN matches `__pin`.
- Implement a method `transfer_funds(amount, target_wallet, entered_pin)`:
 - Check if `verify_pin(entered_pin)` is valid.
 - Check if `__balance >= amount`.
 - Deduct from the sender and deposit into `target_wallet`.
- Test a successful transfer with a correct PIN and a failed transfer with an incorrect PIN.

Case Study 8: University Grading System (Read-Only Transcripts)

A university student portal allows students to view their GPA, but prevents them from modifying course grades directly once calculated.

Write a Python program:

- Create a `StudentTranscript` class with `student_id` (public) and `__grades` (private dictionary mapping subjects to numeric grades).
- Implement `add_grade(subject, grade)` which can only be called internally or at initialization.
- Implement a getter `calculate_gpa()` that converts numerical grades into a weighted GPA average.
- Expose a read-only view method `get_transcript()` that returns a copy of the dictionary so external users cannot mutate internal state via dictionary methods.